

```

import telebot
import logging
import requests
import sqlite3
import time
import threading
from datetime import datetime, timedelta
from telebot.types import LabeledPrice, PreCheckoutQuery

# ===== НАСТРОЙКИ =====
BOT_TOKEN = "8308510677:AAFxv0Q5Er4p-rM30JTrKobgyu4IHBTiXbw"
ADMIN_ID = 6069286437
VPN_PRICE_STARS = 35

# ===== ИНИЦИАЛИЗАЦИЯ =====
bot = telebot.TeleBot(BOT_TOKEN, parse_mode="HTML")
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

# ===== БАЗА ДАННЫХ (SQLite с WAL) =====
def init_db():
    conn = sqlite3.connect('subscriptions.db')
    conn.execute('PRAGMA journal_mode=WAL;') # увеличивает параллельность
    c = conn.cursor()
    c.execute("""CREATE TABLE IF NOT EXISTS users
        (user_id INTEGER PRIMARY KEY,
         username TEXT,
         expires_at TEXT,
         auto_renew INTEGER DEFAULT 1,
         referrer_id INTEGER DEFAULT NULL)""")
    c.execute("""CREATE TABLE IF NOT EXISTS payments
        (payment_id TEXT PRIMARY KEY,
         user_id INTEGER,
         amount INTEGER,
         created_at TEXT)""")
    conn.commit()
    conn.close()

init_db()

# ===== РЕФЕРАЛЬНЫЕ ФУНКЦИИ =====
def get_referral_link(user_id):
    return f"https://t.me/{bot.get_me().username}?start=ref_{user_id}"

def get_referrer(user_id):
    conn = sqlite3.connect('subscriptions.db')
    c = conn.cursor()

```

```

c.execute("SELECT referrer_id FROM users WHERE user_id = ?", (user_id,))
row = c.fetchone()
conn.close()
return row[0] if row else None

```

```

def add_referral_bonus(referrer_id, amount_stars):
    if referrer_id:
        bonus = amount_stars * 0.2
        if bonus >= 1:
            logger.info(f"Бонус {bonus} Stars для {referrer_id}")
            try:
                bot.send_message(referrer_id, f"🎉 +{bonus:.0f} Stars за реферала!")
            except:
                pass

```

===== ПОЛУЧЕНИЕ VPN-КОНФИГА =====

```

VPN_MIRRORS = [
    "https://github.com/nikita29a/FreeProxyList/raw/refs/heads/main/mirror/1.txt",
    "https://github.com/nikita29a/FreeProxyList/raw/refs/heads/main/mirror/2.txt",
    "https://github.com/nikita29a/FreeProxyList/raw/refs/heads/main/mirror/3.txt",
    "https://github.com/nikita29a/FreeProxyList/raw/refs/heads/main/mirror/4.txt",
    "https://github.com/nikita29a/FreeProxyList/raw/refs/heads/main/mirror/5.txt",
]

```

```

def get_vpn_config():
    for url in VPN_MIRRORS:
        try:
            r = requests.get(url, timeout=10)
            if r.status_code == 200:
                lines = r.text.strip().split("\n")
                for line in lines:
                    line = line.strip()
                    if line.startswith(('vless://', 'vmess://', 'trojan://')):
                        logger.info(f"Конфиг получен из {url}")
                        return line
        except Exception as e:
            logger.error(f"Ошибка {url}: {e}")
    return None

```

===== ФУНКЦИИ БАЗЫ =====

```

def save_subscription(user_id, username, expires_at, referrer_id=None):
    conn = sqlite3.connect('subscriptions.db')
    c = conn.cursor()
    c.execute("INSERT OR REPLACE INTO users (user_id, username, expires_at,
auto_renew, referrer_id) VALUES (?, ?, ?, ?, ?)",
        (user_id, username, expires_at.isoformat(), 1, referrer_id))
    conn.commit()
    conn.close()

```

```
def get_subscription(user_id):
    conn = sqlite3.connect('subscriptions.db')
    c = conn.cursor()
    c.execute("SELECT expires_at, auto_renew, referrer_id FROM users WHERE user_id =
?", (user_id,))
    row = c.fetchone()
    conn.close()
    if row:
        expires_at = datetime.fromisoformat(row[0]) if row[0] else None
        return {'expires_at': expires_at, 'auto_renew': bool(row[1]), 'referrer_id': row[2]}
    return None
```

```
def save_payment(payment_id, user_id, amount):
    conn = sqlite3.connect('subscriptions.db')
    c = conn.cursor()
    c.execute("INSERT INTO payments (payment_id, user_id, amount, created_at) VALUES
(?, ?, ?, ?)",
        (payment_id, user_id, amount, datetime.now().isoformat()))
    conn.commit()
    conn.close()
```

===== КОМАНДЫ БОТА =====

```
@bot.message_handler(commands=['start'])
```

```
def start_command(message):
```

```
    user_id = message.from_user.id
```

```
    username = message.from_user.username or message.from_user.first_name
```

```
    args = message.text.split()
```

```
    referrer_id = None
```

```
    if len(args) > 1 and args[1].startswith('ref_'):
```

```
        referrer_id = int(args[1][4:])
```

```
        if referrer_id == user_id:
```

```
            referrer_id = None
```

```
    sub = get_subscription(user_id)
```

```
    if sub and sub['expires_at'] and sub['expires_at'] > datetime.now():
```

```
        days_left = (sub['expires_at'] - datetime.now()).days
```

```
        status_text = f"✅ Активна до {sub['expires_at'].strftime('%d.%m.%Y')} (осталось
{days_left} дн.)"
```

```
    else:
```

```
        status_text = "❌ Нет активной подписки"
```

```
    if not sub:
```

```
        save_subscription(user_id, username, datetime.now() - timedelta(days=1), referrer_id)
```

```
        if referrer_id:
```

```
            bot.send_message(referrer_id, f"🎉 Новый реферал: {username}")
```

```
    ref_link = get_referral_link(user_id)
```

```

bot.reply_to(
    message,
    f"🤖 <b>VPN Shop Bot</b>\n\n"
    f"Привет, {username}!\n"
    f"{status_text}\n\n"
    f"💰 Цена: {VPN_PRICE_STARS} Stars за 30 дней\n\n"
    f"🔑 Команды:\n"
    f"/buy — купить\n"
    f"/status — статус\n"
    f"/ref — рефералка\n"
    f"/cancel — отменить автопродление\n\n"
    f"🔗 Твоя рефералка: {ref_link}\n"
    f"За друга — 20% от его платежа!",
    parse_mode="HTML",
    disable_web_page_preview=True
)

```

```

@bot.message_handler(commands=['buy'])
def buy_command(message):
    user_id = message.from_user.id
    price_amount = VPN_PRICE_STARS * 100
    prices = [LabeledPrice(label="VPN подписка (30 дней)", amount=price_amount)]
    try:
        bot.send_invoice(
            chat_id=message.chat.id,
            title="VPN Подписка",
            description=f"Доступ к VPN на 30 дней\nЦена: {VPN_PRICE_STARS} Stars",
            invoice_payload=f"buy_{user_id}_{int(time.time())}",
            currency="XTR",
            prices=prices,
            start_parameter="vpn_subscription"
        )
        logger.info(f"Инвойс отправлен {user_id}")
    except Exception as e:
        logger.error(f"Ошибка инвойса: {e}")
        bot.reply_to(message, f"❌ Ошибка: {e}")

```

```

@bot.pre_checkout_query_handler(func=lambda query: True)
def process_pre_checkout(pre_checkout_query: PreCheckoutQuery):
    if pre_checkout_query.currency == "XTR":
        bot.answer_pre_checkout_query(pre_checkout_query.id, ok=True)
    else:
        bot.answer_pre_checkout_query(pre_checkout_query.id, ok=False,
        error_message="Только Telegram Stars")

```

```

@bot.message_handler(content_types=['successful_payment'])
def process_successful_payment(message):

```

```

payment_info = message.successful_payment
user_id = message.from_user.id
username = message.from_user.username or message.from_user.first_name
amount_stars = payment_info.total_amount // 100

logger.info(f"Успешная оплата: {user_id} -> {amount_stars} Stars")
save_payment(payment_info.telegram_payment_charge_id, user_id, amount_stars)

referrer = get_referrer(user_id)
if referrer:
    add_referral_bonus(referrer, amount_stars)

vpn_config = get_vpn_config()
if not vpn_config:
    bot.send_message(ADMIN_ID, f"⚠ Нет конфига для {user_id}")
    bot.reply_to(message, "❌ Ошибка получения ключа. Админ уведомлён.")
    return

expires_at = datetime.now() + timedelta(days=30)
save_subscription(user_id, username, expires_at, referrer)

bot.reply_to(
    message,
    f"✅ <b>Оплата прошла!</b>\n\n"
    f"📅 До {expires_at.strftime('%d.%m.%Y')}\n\n"
    f"🔑 <b>Ваш VPN-ключ:</b>\n"
    f"<code>{vpn_config}</code>\n\n"
    f"📱 <b>Как подключиться:</b>\n"
    f"Android: <a"
href='https://play.google.com/store/apps/details?id=com.v2ray.ang'>v2rayNG</a>\n"
    f"iPhone: <a"
href='https://apps.apple.com/app/streisand/id6450535064'>Streisand</a>\n\n"
    f"В приложении: Импорт → Вставить ссылку\n\n"
    f"⚡ Автопродление включено. Отменить: /cancel",
    parse_mode="HTML",
    disable_web_page_preview=True
)
bot.send_message(ADMIN_ID, f"💰 Продажа! @{username} купил подписку за {amount_stars} Stars")

@bot.message_handler(commands=['status'])
def status_command(message):
    user_id = message.from_user.id
    sub = get_subscription(user_id)
    if sub and sub['expires_at'] and sub['expires_at'] > datetime.now():
        days_left = (sub['expires_at'] - datetime.now()).days
        renew = "включено" if sub['auto_renew'] else "отключено"
        bot.reply_to(

```

```

        message,
        f"✅ Активна до {sub['expires_at'].strftime('%d.%m.%Y')} (осталось {days_left}
дн.)\n🔄 Автопродление: {renew}",
        parse_mode="HTML"
    )
else:
    bot.reply_to(message, f"❌ Нет активной подписки. /buy — {VPN_PRICE_STARS}
Stars")

```

```

@bot.message_handler(commands=['ref'])
def ref_command(message):
    user_id = message.from_user.id
    ref_link = get_referral_link(user_id)
    bot.reply_to(message, f"🔗 Твоя рефералка:\n{ref_link}\n\n20% от платежей друзей —
твои!", disable_web_page_preview=True)

```

```

@bot.message_handler(commands=['cancel'])
def cancel_command(message):
    user_id = message.from_user.id
    sub = get_subscription(user_id)
    if not sub or not sub['expires_at'] or sub['expires_at'] < datetime.now():
        bot.reply_to(message, "❌ Нет активной подписки.")
        return
    conn = sqlite3.connect('subscriptions.db')
    c = conn.cursor()
    c.execute("UPDATE users SET auto_renew = 0 WHERE user_id = ?", (user_id,))
    conn.commit()
    conn.close()
    bot.reply_to(message, f"✅ Автопродление отключено. Подписка активна до
{sub['expires_at'].strftime('%d.%m.%Y')}")

```

```

@bot.message_handler(commands=['enable_auto'])
def enable_auto_command(message):
    user_id = message.from_user.id
    sub = get_subscription(user_id)
    if not sub or not sub['expires_at'] or sub['expires_at'] < datetime.now():
        bot.reply_to(message, "❌ Нет активной подписки.")
        return
    conn = sqlite3.connect('subscriptions.db')
    c = conn.cursor()
    c.execute("UPDATE users SET auto_renew = 1 WHERE user_id = ?", (user_id,))
    conn.commit()
    conn.close()
    bot.reply_to(message, "✅ Автопродление включено.")

```

===== АДМИН-КОМАНДЫ =====

```

@bot.message_handler(commands=['stats'])
def stats_command(message):

```

```

if message.from_user.id != ADMIN_ID:
    return
conn = sqlite3.connect('subscriptions.db')
c = conn.cursor()
c.execute("SELECT COUNT(*) FROM users")
total = c.fetchone()[0]
c.execute("SELECT COUNT(*) FROM users WHERE expires_at > ?",
(datetime.now().isoformat(),))
active = c.fetchone()[0]
c.execute("SELECT COUNT(*), SUM(amount) FROM payments")
cnt, amt = c.fetchone()
amt = amt or 0
conn.close()
bot.reply_to(
    message,
    f"

```

```

threshold = datetime.now() + timedelta(days=3)
c.execute("SELECT user_id, expires_at FROM users WHERE expires_at <= ? AND
auto_renew = 1", (threshold.isoformat(),))
rows = c.fetchall()
conn.close()
for user_id, expires_at_str in rows:
    expires_at = datetime.fromisoformat(expires_at_str)
    days_left = (expires_at - datetime.now()).days
    if days_left <= 3 and days_left > 0:
        try:
            prices = [LabeledPrice(label="Продление VPN",
amount=VPN_PRICE_STARS * 100)]
            bot.send_invoice(
                chat_id=user_id,
                title="Продление подписки",
                description=f"Истекает через {days_left} дн. {VPN_PRICE_STARS} Stars
за 30 дней.",
                invoice_payload=f"renew_{user_id}_{int(time.time())}",
                currency="XTR",
                prices=prices,
                start_parameter="vpn_renewal"
            )
            logger.info(f"Счёт на продление отправлен {user_id}")
        except Exception as e:
            logger.error(f"Ошибка продления {user_id}: {e}")
    except Exception as e:
        logger.error(f"Ошибка в авто-проверке: {e}")
    time.sleep(86400)

threading.Thread(target=auto_renew_check, daemon=True).start()

# ===== ЗАПУСК =====
if __name__ == "__main__":
    logger.info("Бот запущен")
    bot.infinity_polling()

```